

Assignment No. 6

Q.1

```
void fun(int n) {
    if (n == 0) {
        return;
    }
    sop(n + " ");
    fun(n-1);
    sop(n+1 + " ");
}
```

o/p

f(4)
f(3)
f(2)
f(1)
f(0)

3
2
1
2
3
4

```
fun(3) {
    if (n == 0) {
        return;
    }
    sop(3)
    fun(2)
    sop(3+1)
    fun(1)
    sop(3+2)
    fun(0)
    sop(3+3)
    return;
}
```

Q.2

```
void fun(int n) {
    if (n == 0) {
        return;
    }
    sop("X");
    fun(n-1);
    sop("Y");
    fun(n-1);
}
```

o/p

X
X
Y
Y
X
Y

f(4)
f(3)
f(2)
f(1)
f(0)
f(1)
f(2)
f(3)
f(4)

```
fun(0) {
    if (n == 0) {
        return;
    }
    sop(0)
    fun(1)
    sop(0+1)
    fun(2)
    sop(0+2)
    fun(3)
    sop(0+3)
    return;
}
```

Recursion Tree

```
fun(0)
├── fun(1)
│   ├── fun(2)
│   │   ├── fun(3)
│   │   │   ├── fun(4)
│   │   │   └── fun(5)
│   │   └── fun(4)
│   └── fun(3)
│       ├── fun(4)
│       └── fun(5)
└── fun(2)
    ├── fun(3)
    └── fun(4)
```

Q.3

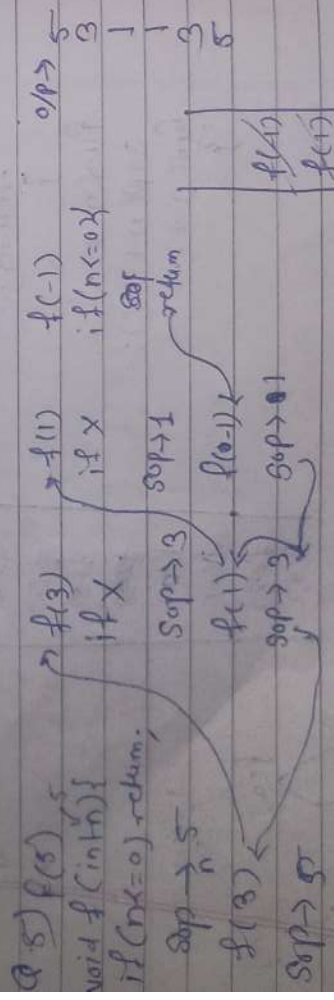
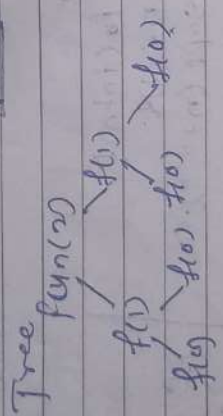
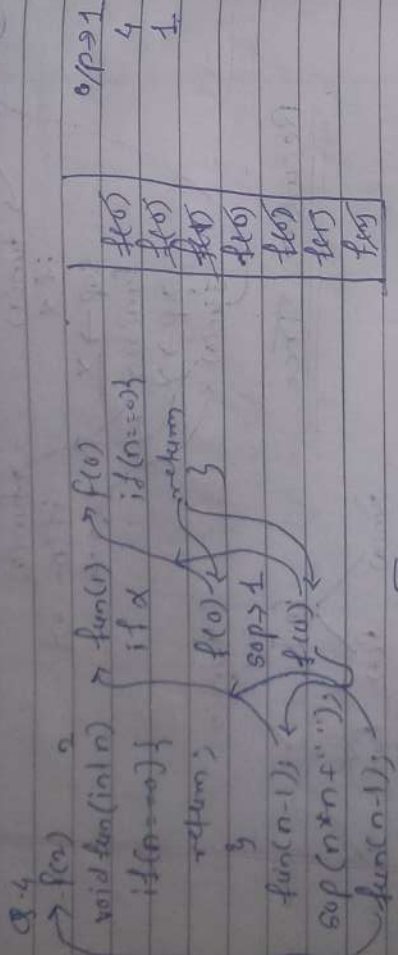
```
void fun(int n) {
    if (n <= 1) {
        return;
    }
    fun(n-2);
    sop(n+1);
    fun(n-2);
}
```

o/p

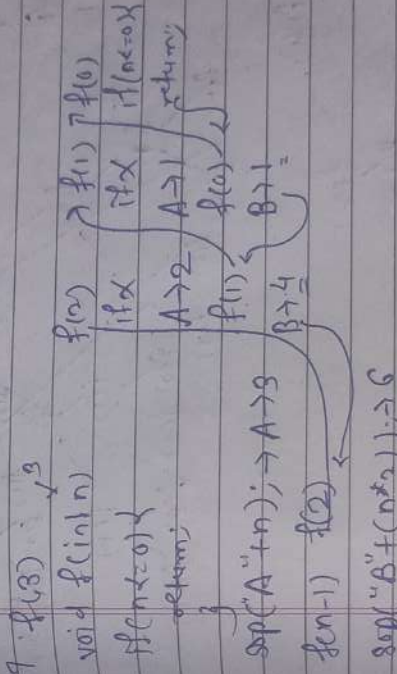
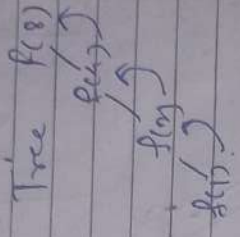
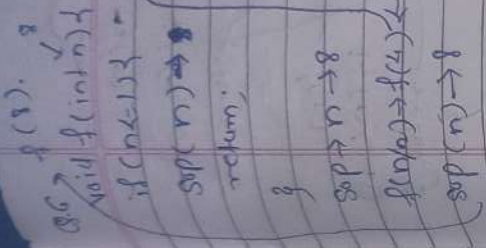
0
2
0
4
0
2
0

f(4)
f(3)
f(2)
f(1)
f(0)
f(1)
f(2)
f(3)
f(4)

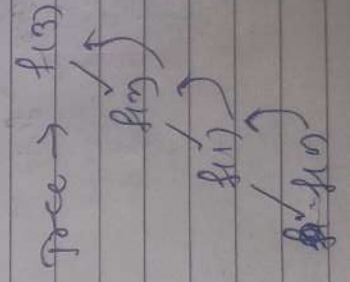
f(0)
f(1)
f(2)
f(3)
f(4)



Recurrence relation: $T(n) = T(n-1) + T(n-1)$



f(0)	f(1)	f(2)	f(3)
1	2	4	8



Handwritten diagram illustrating the recursive calls for calculating the 5th Fibonacci number ($F(5)$).

The diagram shows the sequence of calls and return values:

- $F(5)$ calls $F(4)$ and $F(3)$. It returns 5.
- $F(4)$ calls $F(3)$ and $F(2)$. It returns 3.
- $F(3)$ calls $F(2)$ and $F(1)$. It returns 2.
- $F(2)$ calls $F(1)$ and $F(0)$. It returns 1.
- $F(1)$ returns 1.
- $F(0)$ returns 0.

The diagram is annotated with 'q' and 'q' at various points, and a 'Tree' label at the bottom right.

